

Produktanforderungen (PRD), User Stories & Technisches Konzept für eine Frontend-UI zur Steuerung von Code-Agenten

1. Produkthanforderungsdokument (PRD)

1.1 Vision & Ziele

Vision:Eine zentrale Frontend-UI, die Entwicklern ermöglicht, Code-Agenten über verschiedene IDEs hinweg zu steuern und einen gemeinsamen, synchronisierten Arbeitskontext ("Shared Thread") bereitzustellen.

Ziele:

- **Zentrale Steuerung:** Einheitliche Befehlsausführung für Code-Agenten in verschiedenen IDEs.
- **Kontextmanagement:** Synchronisierter Arbeitskontext (Code-Snippets, Terminal-Output, Notizen).
- **IDE-Integration:** Unterstützung für Cursor (VS Code-basiert) und potenziell weitere IDEs.
- **Echtzeit-Kollaboration:** Nahtlose Zusammenarbeit zwischen Agenten und Entwicklern.

1.2 Scope & Nicht-Scope

In Scope: ✓ Entwicklung einer Web-basierten Frontend-UI mit WebSocket/SRPC-Kommunikation. ✓ Integration mit Cursor IDE über eine dedizierte Erweiterung. ✓ Verwaltung eines "Shared Threads" (Kontextspeicher für Code, Notizen, Terminal-Output). ✓ Unterstützung für natürliche Sprachbefehle und Kontextreferenzierung (@Datei, @Selektion).

Out of Scope: ✗ Direkte Integration mit Windsurf (keine API verfügbar). ✗ Vollständige IDE-Emulation (kein Ersatz für VS Code/Cursor). ✗ Automatische Code-Änderungen ohne Nutzerbestätigung.

1.3 Erfolgskriterien (KPIs)

- **Nutzungsrate:** ≥70% der Cursor-Nutzer aktivieren die Erweiterung.
- **Kontextgenauigkeit:** ≥90% der Befehle erhalten korrekten Kontext.
- **Latenz:** <500ms für Agentenantworten.
- **Stabilität:** 99,9% WebSocket-Verfügbarkeit.

2. User Stories

2.1 Kernfunktionen

ID

User Story

Akzeptanzkriterien

US-001	Als Entwickler möchte ich Code-Agenten über eine zentrale UI steuern, um nicht zwischen IDEs wechseln zu müssen.	✓ UI sendet Befehle an verbundene IDEs. ✓ Antworten werden im Aktivitäts-Feed angezeigt.
--------	--	--

US-002	Als Entwickler möchte ich einen gemeinsamen Kontext (Code, Notizen) für alle Agenten verwalten, um konsistente Ergebnisse zu erhalten.	✓ Kontext wird in Echtzeit synchronisiert. ✓ @Datei -Referenzen funktionieren.
US-003	Als Entwickler möchte ich natürliche Sprachbefehle eingeben, um Code-Agenten zu instruieren.	✓ Agenten verstehen Prompts und liefern ausführbare Lösungen.

2.2 IDE-Integration

ID

User Story

Akzeptanzkriterien

US-004	Als Cursor-Nutzer möchte ich die Frontend-UI über eine Erweiterung verbinden, um Agenten zu nutzen.	✓ Erweiterung stellt WebSocket-Endpunkt bereit. ✓ UI erkennt aktive Cursor-Instanz.
US-005	Als Nutzer möchte ich sehen, welche IDEs verbunden sind, um den Arbeitskontext zu verstehen.	✓ UI zeigt Status (✅ Cursor verbunden / ❌ Windsurf nicht unterstützt).

2.3 Fehlerbehandlung

ID

User Story

Akzeptanzkriterien

US-006	Als Nutzer möchte ich bei Verbindungsabbrüchen benachrichtigt werden, um Probleme zu beheben.	✓ UI zeigt Warnung bei WebSocket-Timeout.
US-007	Als Nutzer möchte ich fehlgeschlagene Agenten-Befehle debuggen können.	✓ Aktivitäts-Feed protokolliert Fehler mit Stacktrace.

3. Technisches Konzept

3.1 Architektur-Übersicht

graph TD; A[Frontend-UI] -->|WebSocket/SRPC| B[Cursor IDE-Erweiterung]; A -->|Shared Thread DB| C[(Kontextspeicher)]; B --> C

3.2 Kernkomponenten

Komponente

Technologie

Verantwortung

Frontend-UI	React, TypeScript	Befehlsingabe, Kontextvisualisierung, Aktivitäts-Feed.
-------------	-------------------	--

Backend-Proxy	Node.js (WebSocket)	Vermittelt zwischen UI und IDE-Erweiterungen.
Shared Thread DB	SQLite (lokal) / Redis	Speichert Code-Snippets, Notizen, Terminal-Output.
Cursor-Erweiterung	VS Code API	Sendet Kontext, empfängt Befehle, führt Code aus.

3.3 Datenfluss

1. **Nutzer gibt Befehl ein** → UI sendet Prompt + Kontext via WebSocket.
1. **Cursor-Erweiterung empfängt Befehl** → führt Aktion aus (z. B. Code-Generierung).
1. **Ergebnis wird zurückgesendet** → UI aktualisiert Aktivitäts-Feed & Kontext-DB.

3.4 Protokolle & APIs

- **WebSocket/SRPC:** Strukturierte RPC-Kommunikation (ähnlich Stagewise).
- **Cursor-API:** Eigene Erweiterung nutzt VS Code API für Lese-/Schreibzugriff.
- **Kontext-API:** REST-Endpunkte für Shared-Thread-DB (CRUD-Operationen).

3.5 Offene Fragen & Risiken

Risiko

Mitigation

Keine Windsurf-Integration	Auf Dokumentation neuer APIs warten oder manuellen Dateiaustausch anbieten.
Latenz bei großen Kontexten	Komprimierung & Delta-Updates für Code-Snippets.

4. Zusammenfassung & nächste Schritte

✅ **PRD definiert** – Klare Ziele, Scope, KPIs. ✅ **User Stories priorisiert** – Fokus auf Cursor-Integration & Kontextmanagement. ✅ **Technisches Konzept erstellt** – WebSocket-basierte Architektur mit Shared-Thread-DB.

Nächste Schritte:

1. **Proof of Concept (PoC):**
 1. Minimal-UI mit WebSocket-Anbindung an Cursor-Erweiterung.
 1. Shared-Thread-DB mit SQLite testen.
1. **Roadmap:**
 1. Phase 1: Cursor-Integration (3 Monate).
 1. Phase 2: Erweiterung auf VS Code (falls gefordert).
 1. Phase 3: Windsurf-Integration (falls API verfügbar).

Anhang:

- [Beispiel-SRPC-Nachrichtenformat](#)
- [Mockups der Frontend-UI](#)

Fragen? Gerne erweitern oder anpassen! 🚀